
Aprendizaje no supervisado en redes neuronales

En esta parte del curso, presentamos modelos de aprendizaje no supervisados en redes neuronales, llamados aprendizaje competitivo y Kohonen SOM. Kohonen SOM fue inventado por el profesor Teuvo Kohonen y es una forma de representar datos multidimensionales en dimensiones mucho más bajas: 1D o 2D. Puede clasificar datos sin supervisión.

El aprendizaje no supervisado tiene como objetivo encontrar patrones ocultos dentro del conjunto de datos y agruparlos en diferentes clases de datos.

Existen muchas técnicas de aprendizaje no supervisadas, a saber, agrupación de K-medias (K-means), reducción de dimensionalidad, EM, etc. La característica común es que no existe una asignación de entrada-salida y solo trabajamos en los valores de entrada para crear un grupo o conjunto de salidas.

Para el caso de las redes neuronales, se pueden utilizar para el aprendizaje no supervisado. Pueden agrupar datos en diferentes cubos (agrupación) o abstraer datos originales en un conjunto diferente de puntos de datos de salida (abstracción de características o reducción de dimensionalidad).

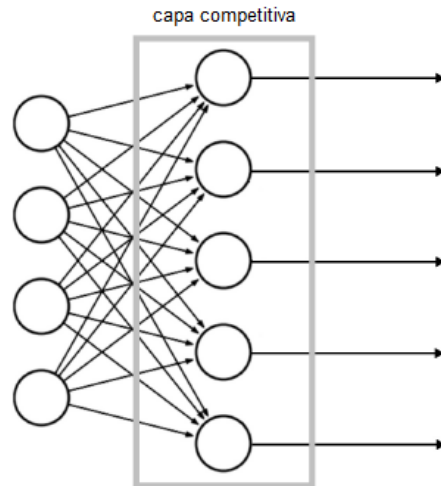
Las técnicas no supervisadas requieren menos potencia de procesamiento y memoria que las técnicas supervisadas.

En redes neuronales no supervisadas, no hay una variable objetivo y no podemos hacer propagación hacia atrás. En cambio, seguimos ajustando los pesos sin la medida del error e intentamos agrupar datos similares. Hay dos métodos que veremos para las redes neuronales sin supervisión:

- Aprendizaje competitivo
- Kohonen SOMs

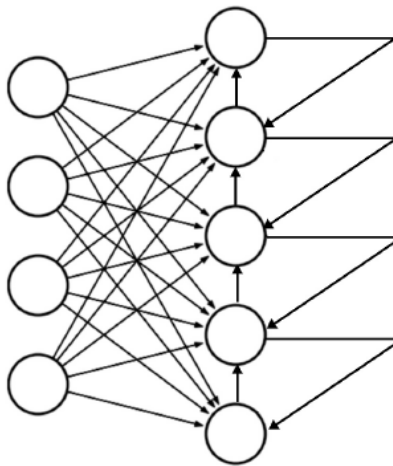
Aprendizaje competitivo

Aquí, los nodos de la red neuronal compiten entre sí por el derecho a responder a un subconjunto de los datos de entrada. La capa oculta se llama capa competitiva. Cada neurona competitiva tiene su propio peso y calculamos la medida de similitud entre el vector de entrada individual y el peso de la neurona. Para cada vector de entrada, las neuronas ocultas compiten entre sí para ver cuál es la más similar al vector de entrada particular:



Se dice que las neuronas de salida compiten por los patrones de entrada.

- Durante el entrenamiento, la neurona de salida que proporciona la activación más alta a un patrón de entrada dado se declara el peso del ganador y se acerca al patrón de entrada, mientras que el resto de las neuronas se mantienen sin cambios.
- Esta estrategia también se llama el ganador se lo lleva todo (winner-takes-all), ya que solo se actualiza la neurona ganadora:



Veamos un ejemplo simple de algoritmo de aprendizaje competitivo para encontrar tres neuronas dentro de los datos de entrada dados:

1. Tendremos tres neuronas de entrada en la capa de entrada. Cada entrada a la neurona es una variable continua y permite que el peso en cada neurona de entrada sea un número aleatorio entre 0.0 y 1.0. La salida de cada nodo es el producto de los tres pesos y su entrada.
2. Cada neurona de la capa competitiva recibe la suma del producto de los pesos y entradas.

3. El nodo de la capa competitiva con el mayor rendimiento se considera el ganador. La entrada se clasifica como dentro del clúster correspondiente a ese nodo.
4. El ganador actualiza cada uno de sus pesos, moviendo el peso de las conexiones que le dieron señales más débiles a las que le dieron señales más fuertes.

Por lo tanto, a medida que recibimos más datos, cada nodo converge en el centro del grupo que ha llegado a representar. Se activa más fuertemente para las entradas que pertenecen a este clúster y más débilmente para las entradas que pertenecen a otros clústeres.

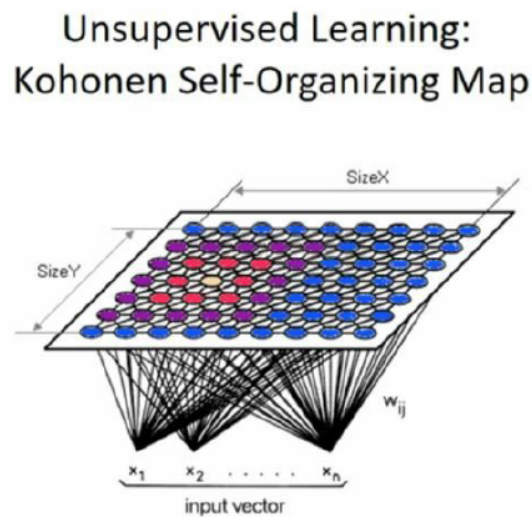
Básicamente, existen dos condiciones de paro del aprendizaje competitivo:

- **Número predefinido de épocas:** solo se ejecutan N épocas y esto evita que el algoritmo se ejecute durante un tiempo relativamente largo sin convergencia
- **Valor mínimo de actualización del peso:** el algoritmo se ejecuta hasta que tengamos un valor mínimo de actualización de peso

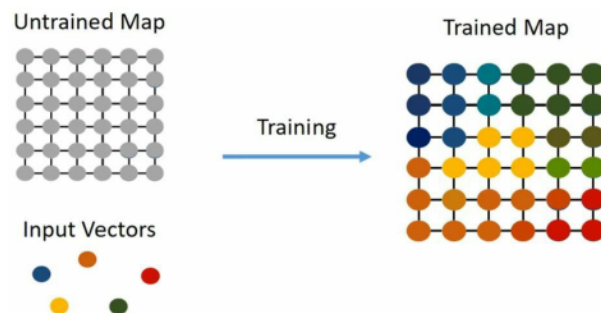
Kohonen SOM

El concepto de aprendizaje competitivo combinado con las neuronas del vecindario nos da Kohonen SOM. Cada neurona en la capa de salida tiene dos vecinos. La neurona que dispara el mayor valor actualiza sus pesos en el aprendizaje competitivo, pero en SOM, las neuronas vecinas también actualizan sus pesos a un ritmo relativamente lento. El número de neuronas vecinas que la red actualiza los pesos se basa en la dimensión del problema.

Para un problema 2D, el SOM se representa de la siguiente manera:



Diagramáticamente, así es como SOM mapea diferentes colores en diferentes grupos:



Comprendamos el funcionamiento de Kohonen SOM paso a paso:

1. El número de entradas y los clústeres que definen la estructura SOM y los pesos de cada nodo se inicializan.
2. Se elige un vector al azar del conjunto de datos de entrenamiento y se presenta a la red.

3. Cada nodo de la red se examina para calcular cuáles son los pesos más similares al vector de entrada. El nodo ganador se conoce comúnmente como la Mejor Unidad de Igualación (Best Matching Unit, BMU).
4. Se calcula el radio de la vecindad de la BMU. Este valor comienza en grande y generalmente se establece como el radio de la red, disminuyendo cada paso de tiempo.
5. Cualquier neurona encontrada dentro del radio de la BMU, calculada en el paso 4, se ajusta para que se parezca más al vector de entrada. Cuanto más cerca esté una neurona de la BMU, más se alterarán sus pesos.
6. Repetir desde el paso 2 para N iteraciones.

Los pasos se repiten para un conjunto de épocas N o hasta que se obtenga una actualización de peso mínimo.

Las SOM se usan en los campos de agrupamiento (agrupación de datos en diferentes segmentos), abstracción de datos (derivando datos de salida de entradas) y reducción de dimensionalidad (reduciendo el número de características de entrada). Las SOM manejan el problema de manera similar a la Escala multidimensional (Multi Dimensional Scaling, MDS), pero en lugar de minimizar las distancias, intentan reagrupar la topología o, en otras palabras, intentan mantener los mismos vecinos.

Veamos un ejemplo de implementación de SOM en R. El paquete kohonen es un paquete que se instalará para usar las funciones ofrecidas en R para SOM.

El siguiente programa R explica algunas funciones del paquete kohonen:

```
> library("kohonen")

> data("wines")
> str(wines)
> head(wines)
> View (wines)

> set.seed(1)
> som.wines = som(scale(wines), grid = somgrid(5, 5, "hexagonal"))
> som.wines
> dim(getCodes(som.wines))

> plot(som.wines, main = "Wine data Kohonen SOM")
> par(mfrow = c(1, 1))
> plot(som.wines, type = "changes", main = "Wine data: SOM")

> training = sample(nrow(wines), 150)
```

```

> Xtraining = scale(wines[training, ])
> Xtest = scale(wines[-training, ],
+             center = attr(Xtraining, "scaled:center"),
+             scale = attr(Xtraining, "scaled:scale"))
> trainingdata = list(measurements = Xtraining,
+                   vintages = vintages[training])
> testdata = list(measurements = Xtest, vintages = vintages[-training])
> mygrid = somgrid(5, 5, "hexagonal")
> som.wines = supersom(trainingdata, grid = mygrid)

> som.prediction = predict(som.wines, newdata = testdata)
> table(vintages[-training], som.prediction$predictions[["vintages"]])

```

El código utiliza un conjunto de datos de vino, que contiene un frame de datos con 177 filas y 13 columnas; las cosechas (vintages) de objetos contienen las etiquetas de clase. Estos datos se obtienen de los análisis químicos de vinos cultivados en la misma región en Italia (Piemonte) pero derivados de tres variedades diferentes, a saber, las uvas Nebbiolo, Barberas y Grignolino.

El vino de la uva Nebbiolo se llama Barolo. Los datos consisten en las cantidades de varios componentes encontrados en cada uno de los tres tipos de vinos, así como algunas variables espectroscópicas.

Ahora, veamos los resultados en cada sección del código.

```
library("kohonen")
```

La primera línea del código es simple, ya que carga la biblioteca que usaremos para cálculos posteriores. Específicamente, la biblioteca de kohonen nos ayudará a entrenar SOM. Además, se admite la interrogación de los mapas y la predicción utilizando mapas entrenados.

Consejo: Recuerda, para instalar una biblioteca que no está presente en la distribución inicial de R, debes usar la función `install.package`. Esta es la función principal para instalar paquetes. Toma un vector de nombres y una biblioteca de destino, descarga los paquetes de los repositorios y los instala.

```

data("wines")
str(wines)
head(wines)
View (wines)

```

Estas líneas cargan el conjunto de datos de vinos (wines), que, como anticipamos, está contenido en la distribución R y lo guarda en un marco de datos denominado data. Luego, usamos la función `str` para ver de forma compacta la estructura del conjunto de datos. La

función *head* se usa para devolver la primera o la última parte del marco de datos. Finalmente, la función de vista (*View*) se utiliza para invocar un visor de datos de estilo de hoja de cálculo en el objeto de frame de datos, como se muestra en la siguiente figura:

	alcohol	malic acid	ash	ash alkalinity	magnesium	tot. phenols	flavonoids	non-flav. phenols	proanth
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82
5	14.20	1.76	2.45	15.2	112	3.27	3.39	0.34	1.97
6	14.39	1.87	2.45	14.6	96	2.50	2.52	0.30	1.98
7	14.06	2.15	2.61	17.6	121	2.60	2.51	0.31	1.25
8	14.83	1.64	2.17	14.0	97	2.80	2.98	0.29	1.98
9	13.86	1.35	2.27	16.0	98	2.98	3.15	0.22	1.85
10	14.10	2.16	2.30	18.0	105	2.95	3.32	0.22	2.38
11	14.12	1.48	2.32	16.8	95	2.20	2.43	0.26	1.57
12	13.75	1.73	2.41	16.0	89	2.60	2.76	0.29	1.81
13	14.75	1.73	2.39	11.4	91	3.10	3.69	0.43	2.81
14	14.38	1.87	2.38	12.0	102	3.30	3.64	0.29	2.96
15	13.63	1.81	2.70	17.2	112	2.85	2.91	0.30	1.46
16	14.30	1.92	2.72	20.0	120	2.80	3.14	0.33	1.97
17	13.83	1.57	2.62	20.0	115	2.95	3.40	0.40	1.72
18	14.19	1.59	2.48	16.5	108	3.30	3.93	0.32	1.86
19	13.64	3.10	2.56	15.2	116	2.70	3.03	0.17	1.66
20	14.06	1.63	2.28	16.0	126	3.00	3.17	0.24	2.10
21	12.93	3.80	2.65	18.6	102	2.41	2.41	0.25	1.98
22	13.71	1.86	2.36	16.6	101	2.61	2.88	0.27	1.69
23	12.85	1.60	2.52	17.8	95	2.48	2.37	0.26	1.46
24	13.50	1.81	2.61	20.0	96	2.53	2.61	0.28	1.66

Seguiremos analizando el código:

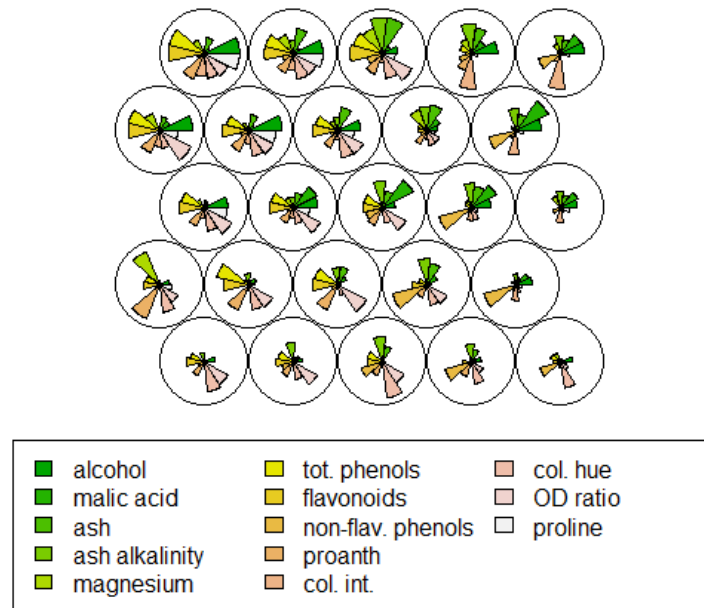
```
set.seed(1)
som.wines = som(scale(wines), grid = somgrid(5, 5, "hexagonal"))
som.wines
dim(getCodes(som.wines))
```

Después de cargar los datos del vino y establecer la semilla (*seed*) para la reproducibilidad, llamamos a *som* para crear una matriz de 5×5 , en la que las características deben agruparse.

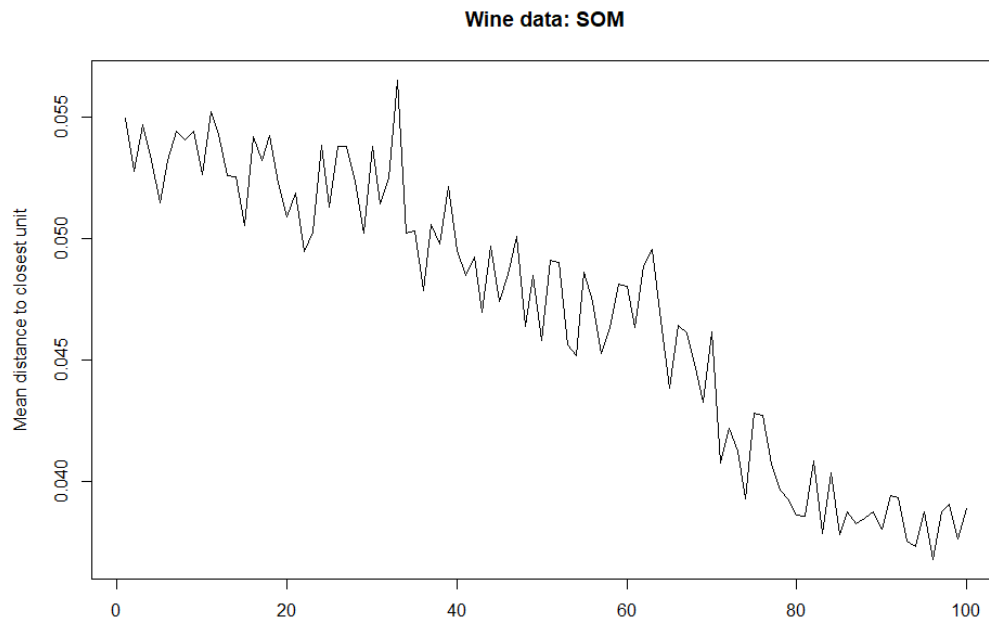
La función realiza internamente el procesamiento de kohonen y el resultado puede verse en los grupos formados con las características. Se han creado 25 clústeres, cada uno de los cuales tiene un conjunto combinado de características que tienen un patrón común, como se muestra en la siguiente imagen:

La siguiente parte del código traza la distancia media a la unidad más cercana frente al número de iteraciones realizadas por som:

```
plot(som.wines, main = "Wine data Kohonen SOM")
par(mfrow = c(1, 1))
plot(som.wines, type = "changes", main = "Wine data: SOM")
```

Wine data Kohonen SOM

En la siguiente figura se muestra la distancia media a la unidad más cercana frente al número de iteraciones:



A continuación, creamos un conjunto de datos de entrenamiento (*training*) con 150 filas y un conjunto de datos de prueba (*test*) con 27 filas. Ejecutamos el SOM y predecimos con los datos de prueba. La función *supersom* se usa aquí. Aquí, el modelo está supervisado por SOM:


```

training = sample(nrow(wines), 150)
Xtraining = scale(wines[training, ])
Xtest = scale(wines[-training, ],
              center = attr(Xtraining, "scaled:center"),
              scale = attr(Xtraining, "scaled:scale"))
trainingdata = list(measurements = Xtraining,
                    vintages = vintages[training])
testdata = list(measurements = Xtest, vintages = vintages[-training])
mygrid = somgrid(5, 5, "hexagonal")
som.wines = supersom(trainingdata, grid = mygrid)

som.prediction = predict(som.wines, newdata = testdata)
table(vintages[-training], som.prediction$predictions[["vintages"]])

```

Finalmente, invocamos la función *table* que usa los factores de clasificación cruzada para construir una tabla de contingencia de los conteos en cada combinación de niveles de factores, como se muestra a continuación:

```

> table(vintages[-training], som.prediction$predictions[["vintages"]])

```

	Barbera	Barolo	Grignolino
Barbera	4	0	0
Barolo	0	12	0
Grignolino	0	0	11

```

> |

```

El paquete *kohonen* presenta SOM estándar y dos extensiones: para fines de clasificación y regresión, y para minería de datos. Además, tiene una amplia capacidad gráfica para la visualización.

Para conocer las funciones disponibles en el paquete *kohonen*, puedes consultar la descripción en línea que existe en el siguiente sitio:

<https://cran.r-project.org/web/packages/kohonen/kohonen.pdf>